

ODEX

ENTERPRISE AUTONOMOUS AI BUILDER COMPLETE TECHNICAL BLUEPRINT

[VERSION 1.5.0-ALPHA-FINAL]

[TOTAL CORES: 50]

[STATUS: ENGINEERING MASTER SPEC]

SECTION 0 — EXECUTIVE SUMMARY

This document constitutes the authoritative technical specification for the ODEX Autonomous AI Builder. ODEX is a decentralized, multimodal software factory that utilizes a swarm of 50+ specialized intelligence cores to transform human intent into production-ready software systems.

Primary Goal: To decouple the "What" from the "How", allowing human operators to manage software ecosystems at the level of pure intent while the machine handles architecture, generation, debugging, testing, and deployment.

SECTION 1 — THE ODEX WORKSPACE

MULTIPURPOSE ENGINEERING INTERFACE

LEFT PANEL: AI SWARM & INTELLIGENCE CONTEXT

Detailed visualization of the "Active Brain Swarm". Displays which cores are currently processing, token metrics, and reasoning traces. Includes the **Instruction Injection Port** for real-time human nudging.

MIDDLE PANEL: LIVE BUILD & LOG ENGINE

The core execution area. Features a specialized code editor displaying "GHOST FILES"—files currently being hallucinated and formalized by the AI. Above the editor is the **Pipeline Tracker** showing atomic stage transitions (Plan → Build → Fix → Test).

RIGHT PANEL: LIVE PREVIEW & INTERACTION

A full-stack sandbox environment. Users can interact with the app in real-time. Changes made by the AI are "Hot-Swapped" into the preview within <250ms.

```
// WORKSPACE STATE DEFINITION (C-10)
{
  "workspace": {
    "active_project": "PROJECT-771",
    "panels": {
      "left": { "agent_feed": true, "context_stack": ["React", "Stripe", "Auth0"] },
      "middle": { "editor_mode": "diff_view", "live_logs": "STDOUT_STREAM" },
      "right": { "preview_url": "http://sandbox-env-88.odex.local", "hot_reload": "AUTO" }
    }
  }
}
```

SECTION 2 — THE MULTIMODAL HUMAN LOOP

The Human Loop (HITL) is the central quality and taste arbiter of the workspace. Unlike standard automation, ODEX utilizes **Strategic Insertion Gates**.

1. PLANNING & INTENT LOOP

Occurs after C-02 (Pilot) decomposes the prompt. The human reviews the proposed "Task Tree" and adjusts priorities or technical selections before build begins.

2. ARCHITECTURE APPROVAL LOOP

C-06 (Architect) presents the Database ERD and Service Map. Human operator provides "Go/No-Go" logic for critical infrastructure decisions.

3. AESTHETIC & UI LOOP

AI generates 3 variation options for the UI layer based on C-11. Human selects the "Soul" of the project via high-fidelity selection.

4. DEBUGGING & CORRECTION LOOP

Triggered when C-08 (Fixer) reaches a logic impasse. The system presents the conflict to the human with "Suggested Resolutions".

SECTION 3 — THE 50 INTEGRATED CORES

Every core in ODEX is a self-contained intelligence agent with a specific mission, contract, and logic loop.

CORE MISSION

High-fidelity multimodal intent capture.

INPUT SIGNAL	INTERNAL LOGIC FLOW	OUTPUT ARTIFACT
Context Vector + Intent	Tokenization → Intent Classification → Context Hydration.	Validated Block

Dependencies

C-02 (Pilot), C-05 (State), C-18 (Task Queue), System Bus.

Core Execution Code Snippet

```
// Logic for User Input Engine (C-01)
async function executeCore_C_01(context, task) {
  const pilot = await Pilot.getChannel();
  const state = await StateCore.snapshot(task.id);

  // Core Logic: Tokenization → Intent Classification → Context Hydration.
  const result = await Intelligence.process(core_definition, context);

  if (result.confidence > 0.95) {
    return await BuildEngine.commit(result.payload);
  } else {
    return await HumanLoop.trigger("VERIFY_OUTPUT", result);
  }
}
```


CORE MISSION

Lead orchestrator and task decomposer.

INPUT SIGNAL	INTERNAL LOGIC FLOW	OUTPUT ARTIFACT
Context Vector + Intent	Strategic planning → Swarm distribution → Feedback loop management.	Validated Block

Dependencies

C-02 (Pilot), C-05 (State), C-18 (Task Queue), System Bus.

Core Execution Code Snippet

```
// Logic for AI Pilot (Master Brain) (C-02)
async function executeCore_C_02(context, task) {
  const pilot = await Pilot.getChannel();
  const state = await StateCore.snapshot(task.id);

  // Core Logic: Strategic planning → Swarm distribution → Feedback loop management.
  const result = await Intelligence.process(core_definition, context);

  if (result.confidence > 0.95) {
    return await BuildEngine.commit(result.payload);
  } else {
    return await HumanLoop.trigger("VERIFY_OUTPUT", result);
  }
}
```

CORE MISSION

Dynamic template generation for worker context.

INPUT SIGNAL	INTERNAL LOGIC FLOW	OUTPUT ARTIFACT
Context Vector + Intent	Constraint injection → Few-shot formatting → Safety wrapping.	Validated Block

Dependencies

C-02 (Pilot), C-05 (State), C-18 (Task Queue), System Bus.

Core Execution Code Snippet

```
// Logic for Prompt Engineering Core (C-03)
async function executeCore_C_03(context, task) {
  const pilot = await Pilot.getChannel();
  const state = await StateCore.snapshot(task.id);

  // Core Logic: Constraint injection → Few-shot formatting → Safety wrapping.
  const result = await Intelligence.process(core_definition, context);

  if (result.confidence > 0.95) {
    return await BuildEngine.commit(result.payload);
  } else {
    return await HumanLoop.trigger("VERIFY_OUTPUT", result);
  }
}
```

CORE MISSION

Pub/Sub backbone for cross-core comms.

INPUT SIGNAL	INTERNAL LOGIC FLOW	OUTPUT ARTIFACT
Context Vector + Intent	Event sourcing → Bus arbitration → Delivery confirmation.	Validated Block

Dependencies

C-02 (Pilot), C-05 (State), C-18 (Task Queue), System Bus.

Core Execution Code Snippet

```
// Logic for Event System Core (C-04)
async function executeCore_C_04(context, task) {
  const pilot = await Pilot.getChannel();
  const state = await StateCore.snapshot(task.id);

  // Core Logic: Event sourcing → Bus arbitration → Delivery confirmation.
  const result = await Intelligence.process(core_definition, context);

  if (result.confidence > 0.95) {
    return await BuildEngine.commit(result.payload);
  } else {
    return await HumanLoop.trigger("VERIFY_OUTPUT", result);
  }
}
```

CORE MISSION

Singleton store for active build state.

INPUT SIGNAL	INTERNAL LOGIC FLOW	OUTPUT ARTIFACT
Context Vector + Intent	Snapshotting → Rollback history → Distributed consistency.	Validated Block

Dependencies

C-02 (Pilot), C-05 (State), C-18 (Task Queue), System Bus.

Core Execution Code Snippet

```
// Logic for State Management Core (C-05)
async function executeCore_C_05(context, task) {
  const pilot = await Pilot.getChannel();
  const state = await StateCore.snapshot(task.id);

  // Core Logic: Snapshotting → Rollback history → Distributed consistency.
  const result = await Intelligence.process(core_definition, context);

  if (result.confidence > 0.95) {
    return await BuildEngine.commit(result.payload);
  } else {
    return await HumanLoop.trigger("VERIFY_OUTPUT", result);
  }
}
```

CORE MISSION

Structural scaffolding and tech-stack mapping.

INPUT SIGNAL	INTERNAL LOGIC FLOW	OUTPUT ARTIFACT
Context Vector + Intent	Constraint review → Model selection → ERD generation.	Validated Block

Dependencies

C-02 (Pilot), C-05 (State), C-18 (Task Queue), System Bus.

Core Execution Code Snippet

```
// Logic for Architecture Planner (C-06)
async function executeCore_C_06(context, task) {
  const pilot = await Pilot.getChannel();
  const state = await StateCore.snapshot(task.id);

  // Core Logic: Constraint review → Model selection → ERD generation.
  const result = await Intelligence.process(core_definition, context);

  if (result.confidence > 0.95) {
    return await BuildEngine.commit(result.payload);
  } else {
    return await HumanLoop.trigger("VERIFY_OUTPUT", result);
  }
}
```

CORE MISSION

Atomic file generation and patching.

INPUT SIGNAL	INTERNAL LOGIC FLOW	OUTPUT ARTIFACT
Context Vector + Intent	Snippet retrieval → Diff application → Tree writing.	Validated Block

Dependencies

C-02 (Pilot), C-05 (State), C-18 (Task Queue), System Bus.

Core Execution Code Snippet

```
// Logic for Code Builder Engine (C-07)
async function executeCore_C_07(context, task) {
  const pilot = await Pilot.getChannel();
  const state = await StateCore.snapshot(task.id);

  // Core Logic: Snippet retrieval → Diff application → Tree writing.
  const result = await Intelligence.process(core_definition, context);

  if (result.confidence > 0.95) {
    return await BuildEngine.commit(result.payload);
  } else {
    return await HumanLoop.trigger("VERIFY_OUTPUT", result);
  }
}
```

CORE MISSION

Self-healing loop for runtime/build errors.

INPUT SIGNAL	INTERNAL LOGIC FLOW	OUTPUT ARTIFACT
Context Vector + Intent	Log scraping → Trace mapping → Remediation generation.	Validated Block

Dependencies

C-02 (Pilot), C-05 (State), C-18 (Task Queue), System Bus.

Core Execution Code Snippet

```
// Logic for Debugger & Fixer (C-08)
async function executeCore_C_08(context, task) {
  const pilot = await Pilot.getChannel();
  const state = await StateCore.snapshot(task.id);

  // Core Logic: Log scraping → Trace mapping → Remediation generation.
  const result = await Intelligence.process(core_definition, context);

  if (result.confidence > 0.95) {
    return await BuildEngine.commit(result.payload);
  } else {
    return await HumanLoop.trigger("VERIFY_OUTPUT", result);
  }
}
```

CORE MISSION

Automatic test suite generation.

INPUT SIGNAL	INTERNAL LOGIC FLOW	OUTPUT ARTIFACT
Context Vector + Intent	Signature extraction → Mock injection → Coverage analysis.	Validated Block

Dependencies

C-02 (Pilot), C-05 (State), C-18 (Task Queue), System Bus.

Core Execution Code Snippet

```
// Logic for Unit Testing Core (C-09)
async function executeCore_C_09(context, task) {
  const pilot = await Pilot.getChannel();
  const state = await StateCore.snapshot(task.id);

  // Core Logic: Signature extraction → Mock injection → Coverage analysis.
  const result = await Intelligence.process(core_definition, context);

  if (result.confidence > 0.95) {
    return await BuildEngine.commit(result.payload);
  } else {
    return await HumanLoop.trigger("VERIFY_OUTPUT", result);
  }
}
```


CORE MISSION

Ensuring inter-service coherence.

INPUT SIGNAL	INTERNAL LOGIC FLOW	OUTPUT ARTIFACT
Context Vector + Intent	Flow mapping → Endpoint simulation → Status verification.	Validated Block

Dependencies

C-02 (Pilot), C-05 (State), C-18 (Task Queue), System Bus.

Core Execution Code Snippet

```
// Logic for Integration Testing Core (C-10)
async function executeCore_C_10(context, task) {
  const pilot = await Pilot.getChannel();
  const state = await StateCore.snapshot(task.id);

  // Core Logic: Flow mapping → Endpoint simulation → Status verification.
  const result = await Intelligence.process(core_definition, context);

  if (result.confidence > 0.95) {
    return await BuildEngine.commit(result.payload);
  } else {
    return await HumanLoop.trigger("VERIFY_OUTPUT", result);
  }
}
```

CORE MISSION

Aesthetic refinement and asset generation.

INPUT SIGNAL	INTERNAL LOGIC FLOW	OUTPUT ARTIFACT
Context Vector + Intent	Design system lookup → Theme injection → View composition.	Validated Block

Dependencies

C-02 (Pilot), C-05 (State), C-18 (Task Queue), System Bus.

Core Execution Code Snippet

```
// Logic for UI/UX Design Engine (C-11)
async function executeCore_C_11(context, task) {
  const pilot = await Pilot.getChannel();
  const state = await StateCore.snapshot(task.id);

  // Core Logic: Design system lookup → Theme injection → View composition.
  const result = await Intelligence.process(core_definition, context);

  if (result.confidence > 0.95) {
    return await BuildEngine.commit(result.payload);
  } else {
    return await HumanLoop.trigger("VERIFY_OUTPUT", result);
  }
}
```

CORE MISSION

Relational and NoSQL structural design.

INPUT SIGNAL	INTERNAL LOGIC FLOW	OUTPUT ARTIFACT
Context Vector + Intent	Entity relation mapping → Index optimization → Migration gen.	Validated Block

Dependencies

C-02 (Pilot), C-05 (State), C-18 (Task Queue), System Bus.

Core Execution Code Snippet

```
// Logic for Database Schema Core (C-12)
async function executeCore_C_12(context, task) {
  const pilot = await Pilot.getChannel();
  const state = await StateCore.snapshot(task.id);

  // Core Logic: Entity relation mapping → Index optimization → Migration gen.
  const result = await Intelligence.process(core_definition, context);

  if (result.confidence > 0.95) {
    return await BuildEngine.commit(result.payload);
  } else {
    return await HumanLoop.trigger("VERIFY_OUTPUT", result);
  }
}
```

CORE MISSION

REST/GraphQL/gRPC contract management.

INPUT SIGNAL	INTERNAL LOGIC FLOW	OUTPUT ARTIFACT
Context Vector + Intent	Contract definition → Documentation gen → Endpoint logic.	Validated Block

Dependencies

C-02 (Pilot), C-05 (State), C-18 (Task Queue), System Bus.

Core Execution Code Snippet

```
// Logic for API Orchestrator (C-13)
async function executeCore_C_13(context, task) {
  const pilot = await Pilot.getChannel();
  const state = await StateCore.snapshot(task.id);

  // Core Logic: Contract definition → Documentation gen → Endpoint logic.
  const result = await Intelligence.process(core_definition, context);

  if (result.confidence > 0.95) {
    return await BuildEngine.commit(result.payload);
  } else {
    return await HumanLoop.trigger("VERIFY_OUTPUT", result);
  }
}
```

CORE MISSION

Auth pattern and penetration prevention.

INPUT SIGNAL	INTERNAL LOGIC FLOW	OUTPUT ARTIFACT
Context Vector + Intent	Audit trail → Pattern scanning → Credential vaulting.	Validated Block

Dependencies

C-02 (Pilot), C-05 (State), C-18 (Task Queue), System Bus.

Core Execution Code Snippet

```
// Logic for Security Hardening Core (C-14)
async function executeCore_C_14(context, task) {
  const pilot = await Pilot.getChannel();
  const state = await StateCore.snapshot(task.id);

  // Core Logic: Audit trail → Pattern scanning → Credential vaulting.
  const result = await Intelligence.process(core_definition, context);

  if (result.confidence > 0.95) {
    return await BuildEngine.commit(result.payload);
  } else {
    return await HumanLoop.trigger("VERIFY_OUTPUT", result);
  }
}
```

CORE MISSION

Automated cloud orchestration.

INPUT SIGNAL	INTERNAL LOGIC FLOW	OUTPUT ARTIFACT
Context Vector + Intent	Dockerization → Env injection → Release rotation.	Validated Block

Dependencies

C-02 (Pilot), C-05 (State), C-18 (Task Queue), System Bus.

Core Execution Code Snippet

```
// Logic for CI/CD Deployment Core (C-15)
async function executeCore_C_15(context, task) {
  const pilot = await Pilot.getChannel();
  const state = await StateCore.snapshot(task.id);

  // Core Logic: Dockerization → Env injection → Release rotation.
  const result = await Intelligence.process(core_definition, context);

  if (result.confidence > 0.95) {
    return await BuildEngine.commit(result.payload);
  } else {
    return await HumanLoop.trigger("VERIFY_OUTPUT", result);
  }
}
```

CORE MISSION

Semantic retrieval and RAG management.

INPUT SIGNAL	INTERNAL LOGIC FLOW	OUTPUT ARTIFACT
Context Vector + Intent	Vector embedding → Semantic search → Context injection.	Validated Block

Dependencies

C-02 (Pilot), C-05 (State), C-18 (Task Queue), System Bus.

Core Execution Code Snippet

```
// Logic for Memory Pipeline Core (C-16)
async function executeCore_C_16(context, task) {
  const pilot = await Pilot.getChannel();
  const state = await StateCore.snapshot(task.id);

  // Core Logic: Vector embedding → Semantic search → Context injection.
  const result = await Intelligence.process(core_definition, context);

  if (result.confidence > 0.95) {
    return await BuildEngine.commit(result.payload);
  } else {
    return await HumanLoop.trigger("VERIFY_OUTPUT", result);
  }
}
```

CORE MISSION

Dynamic preference capture and tuning.

INPUT SIGNAL	INTERNAL LOGIC FLOW	OUTPUT ARTIFACT
Context Vector + Intent	Interaction logging → Weight adjustment → User-model update.	Validated Block

Dependencies

C-02 (Pilot), C-05 (State), C-18 (Task Queue), System Bus.

Core Execution Code Snippet

```
// Logic for Learn & Go Core (C-17)
async function executeCore_C_17(context, task) {
  const pilot = await Pilot.getChannel();
  const state = await StateCore.snapshot(task.id);

  // Core Logic: Interaction logging → Weight adjustment → User-model update.
  const result = await Intelligence.process(core_definition, context);

  if (result.confidence > 0.95) {
    return await BuildEngine.commit(result.payload);
  } else {
    return await HumanLoop.trigger("VERIFY_OUTPUT", result);
  }
}
```


CORE MISSION

High-throughput execution scheduling.

INPUT SIGNAL	INTERNAL LOGIC FLOW	OUTPUT ARTIFACT
Context Vector + Intent	Priority sorting → Worker affinity → Retry backoff.	Validated Block

Dependencies

C-02 (Pilot), C-05 (State), C-18 (Task Queue), System Bus.

Core Execution Code Snippet

```
// Logic for Task Queue Core (C-18)
async function executeCore_C_18(context, task) {
  const pilot = await Pilot.getChannel();
  const state = await StateCore.snapshot(task.id);

  // Core Logic: Priority sorting → Worker affinity → Retry backoff.
  const result = await Intelligence.process(core_definition, context);

  if (result.confidence > 0.95) {
    return await BuildEngine.commit(result.payload);
  } else {
    return await HumanLoop.trigger("VERIFY_OUTPUT", result);
  }
}
```

CORE MISSION

Extensibility layer for 3rd party tools.

INPUT SIGNAL	INTERNAL LOGIC FLOW	OUTPUT ARTIFACT
Context Vector + Intent	Interface definition → Sandbox execution → Hook management.	Validated Block

Dependencies

C-02 (Pilot), C-05 (State), C-18 (Task Queue), System Bus.

Core Execution Code Snippet

```
// Logic for Plugin System Core (C-19)
async function executeCore_C_19(context, task) {
  const pilot = await Pilot.getChannel();
  const state = await StateCore.snapshot(task.id);

  // Core Logic: Interface definition → Sandbox execution → Hook management.
  const result = await Intelligence.process(core_definition, context);

  if (result.confidence > 0.95) {
    return await BuildEngine.commit(result.payload);
  } else {
    return await HumanLoop.trigger("VERIFY_OUTPUT", result);
  }
}
```

CORE MISSION

Telemetry tracking and usage insights.

INPUT SIGNAL	INTERNAL LOGIC FLOW	OUTPUT ARTIFACT
Context Vector + Intent	Event aggregation → Metric calculation → Report gen.	Validated Block

Dependencies

C-02 (Pilot), C-05 (State), C-18 (Task Queue), System Bus.

Core Execution Code Snippet

```
// Logic for Analytics & BI Core (C-20)
async function executeCore_C_20(context, task) {
  const pilot = await Pilot.getChannel();
  const state = await StateCore.snapshot(task.id);

  // Core Logic: Event aggregation → Metric calculation → Report gen.
  const result = await Intelligence.process(core_definition, context);

  if (result.confidence > 0.95) {
    return await BuildEngine.commit(result.payload);
  } else {
    return await HumanLoop.trigger("VERIFY_OUTPUT", result);
  }
}
```

CORE MISSION

Data redundancy and multi-device sync.

INPUT SIGNAL	INTERNAL LOGIC FLOW	OUTPUT ARTIFACT
Context Vector + Intent	Differential backup → Checksum verification → Restore logic.	Validated Block

Dependencies

C-02 (Pilot), C-05 (State), C-18 (Task Queue), System Bus.

Core Execution Code Snippet

```
// Logic for Backup & Sync Core (C-21)
async function executeCore_C_21(context, task) {
  const pilot = await Pilot.getChannel();
  const state = await StateCore.snapshot(task.id);

  // Core Logic: Differential backup → Checksum verification → Restore logic.
  const result = await Intelligence.process(core_definition, context);

  if (result.confidence > 0.95) {
    return await BuildEngine.commit(result.payload);
  } else {
    return await HumanLoop.trigger("VERIFY_OUTPUT", result);
  }
}
```

CORE MISSION

Critical system survival in event of failure.

INPUT SIGNAL	INTERNAL LOGIC FLOW	OUTPUT ARTIFACT
Context Vector + Intent	Circuit breaking → Degradation mode → Alerting.	Validated Block

Dependencies

C-02 (Pilot), C-05 (State), C-18 (Task Queue), System Bus.

Core Execution Code Snippet

```
// Logic for Failsafe & Sentry Core (C-22)
async function executeCore_C_22(context, task) {
  const pilot = await Pilot.getChannel();
  const state = await StateCore.snapshot(task.id);

  // Core Logic: Circuit breaking → Degradation mode → Alerting.
  const result = await Intelligence.process(core_definition, context);

  if (result.confidence > 0.95) {
    return await BuildEngine.commit(result.payload);
  } else {
    return await HumanLoop.trigger("VERIFY_OUTPUT", result);
  }
}
```

CORE MISSION

Manage compilers, runtimes, and dependencies.

INPUT SIGNAL	INTERNAL LOGIC FLOW	OUTPUT ARTIFACT
Context Vector + Intent	Version resolution → Environment isolation → Build automation.	Validated Block

Dependencies

C-02 (Pilot), C-05 (State), C-18 (Task Queue), System Bus.

Core Execution Code Snippet

```
// Logic for Toolchain Core (C-23)
async function executeCore_C_23(context, task) {
  const pilot = await Pilot.getChannel();
  const state = await StateCore.snapshot(task.id);

  // Core Logic: Version resolution → Environment isolation → Build automation.
  const result = await Intelligence.process(core_definition, context);

  if (result.confidence > 0.95) {
    return await BuildEngine.commit(result.payload);
  } else {
    return await HumanLoop.trigger("VERIFY_OUTPUT", result);
  }
}
```

CORE MISSION

Self-updating system for core engines.

INPUT SIGNAL	INTERNAL LOGIC FLOW	OUTPUT ARTIFACT
Context Vector + Intent	Version check → Atomic update → Regression check.	Validated Block

Dependencies

C-02 (Pilot), C-05 (State), C-18 (Task Queue), System Bus.

Core Execution Code Snippet

```
// Logic for Update & Patch Core (C-24)
async function executeCore_C_24(context, task) {
  const pilot = await Pilot.getChannel();
  const state = await StateCore.snapshot(task.id);

  // Core Logic: Version check → Atomic update → Regression check.
  const result = await Intelligence.process(core_definition, context);

  if (result.confidence > 0.95) {
    return await BuildEngine.commit(result.payload);
  } else {
    return await HumanLoop.trigger("VERIFY_OUTPUT", result);
  }
}
```

CORE MISSION

Automatic technical and user manual gen.

INPUT SIGNAL	INTERNAL LOGIC FLOW	OUTPUT ARTIFACT
Context Vector + Intent	Code analysis → JSDoc/Swagger extraction → Markdown gen.	Validated Block

Dependencies

C-02 (Pilot), C-05 (State), C-18 (Task Queue), System Bus.

Core Execution Code Snippet

```
// Logic for Documentation Engine (C-25)
async function executeCore_C_25(context, task) {
  const pilot = await Pilot.getChannel();
  const state = await StateCore.snapshot(task.id);

  // Core Logic: Code analysis → JSDoc/Swagger extraction → Markdown gen.
  const result = await Intelligence.process(core_definition, context);

  if (result.confidence > 0.95) {
    return await BuildEngine.commit(result.payload);
  } else {
    return await HumanLoop.trigger("VERIFY_OUTPUT", result);
  }
}
```


CORE MISSION

WCAG compliance and ARIA generation.

INPUT SIGNAL	INTERNAL LOGIC FLOW	OUTPUT ARTIFACT
Context Vector + Intent	Audit loop → Hint injection → Screen-reader optimize.	Validated Block

Dependencies

C-02 (Pilot), C-05 (State), C-18 (Task Queue), System Bus.

Core Execution Code Snippet

```
// Logic for Accessibility Core (C-26)
async function executeCore_C_26(context, task) {
  const pilot = await Pilot.getChannel();
  const state = await StateCore.snapshot(task.id);

  // Core Logic: Audit loop → Hint injection → Screen-reader optimize.
  const result = await Intelligence.process(core_definition, context);

  if (result.confidence > 0.95) {
    return await BuildEngine.commit(result.payload);
  } else {
    return await HumanLoop.trigger("VERIFY_OUTPUT", result);
  }
}
```

CORE MISSION

Optimizing search visibility and load speeds.

INPUT SIGNAL	INTERNAL LOGIC FLOW	OUTPUT ARTIFACT
Context Vector + Intent	Metadata injection → Asset compression → Sitemap gen.	Validated Block

Dependencies

C-02 (Pilot), C-05 (State), C-18 (Task Queue), System Bus.

Core Execution Code Snippet

```
// Logic for SEO & Performance Core (C-27)
async function executeCore_C_27(context, task) {
  const pilot = await Pilot.getChannel();
  const state = await StateCore.snapshot(task.id);

  // Core Logic: Metadata injection → Asset compression → Sitemap gen.
  const result = await Intelligence.process(core_definition, context);

  if (result.confidence > 0.95) {
    return await BuildEngine.commit(result.payload);
  } else {
    return await HumanLoop.trigger("VERIFY_OUTPUT", result);
  }
}
```

CORE MISSION

Converting web builds to PWA/Native.

INPUT SIGNAL	INTERNAL LOGIC FLOW	OUTPUT ARTIFACT
Context Vector + Intent	Platform mapping → Wrapper gen → Build adjustment.	Validated Block

Dependencies

C-02 (Pilot), C-05 (State), C-18 (Task Queue), System Bus.

Core Execution Code Snippet

```
// Logic for Mobile Porter Core (C-28)
async function executeCore_C_28(context, task) {
  const pilot = await Pilot.getChannel();
  const state = await StateCore.snapshot(task.id);

  // Core Logic: Platform mapping → Wrapper gen → Build adjustment.
  const result = await Intelligence.process(core_definition, context);

  if (result.confidence > 0.95) {
    return await BuildEngine.commit(result.payload);
  } else {
    return await HumanLoop.trigger("VERIFY_OUTPUT", result);
  }
}
```

CORE MISSION

Legal and regulatory alignment (GDPR/HIPAA).

INPUT SIGNAL	INTERNAL LOGIC FLOW	OUTPUT ARTIFACT
Context Vector + Intent	Policy mapping → Data restriction → Audit logging.	Validated Block

Dependencies

C-02 (Pilot), C-05 (State), C-18 (Task Queue), System Bus.

Core Execution Code Snippet

```
// Logic for Compliance AI (C-29)
async function executeCore_C_29(context, task) {
  const pilot = await Pilot.getChannel();
  const state = await StateCore.snapshot(task.id);

  // Core Logic: Policy mapping → Data restriction → Audit logging.
  const result = await Intelligence.process(core_definition, context);

  if (result.confidence > 0.95) {
    return await BuildEngine.commit(result.payload);
  } else {
    return await HumanLoop.trigger("VERIFY_OUTPUT", result);
  }
}
```

CORE MISSION

Syncing concurrent multi-agent code edits.

INPUT SIGNAL	INTERNAL LOGIC FLOW	OUTPUT ARTIFACT
Context Vector + Intent	Diff analysis → Merge strategies → Human escalations.	Validated Block

Dependencies

C-02 (Pilot), C-05 (State), C-18 (Task Queue), System Bus.

Core Execution Code Snippet

```
// Logic for Conflict Resolver (C-30)
async function executeCore_C_30(context, task) {
  const pilot = await Pilot.getChannel();
  const state = await StateCore.snapshot(task.id);

  // Core Logic: Diff analysis → Merge strategies → Human escalations.
  const result = await Intelligence.process(core_definition, context);

  if (result.confidence > 0.95) {
    return await BuildEngine.commit(result.payload);
  } else {
    return await HumanLoop.trigger("VERIFY_OUTPUT", result);
  }
}
```

CORE MISSION

Automated PR reviews and style checks.

INPUT SIGNAL	INTERNAL LOGIC FLOW	OUTPUT ARTIFACT
Context Vector + Intent	Static analysis → Sentiment check → Hint generation.	Validated Block

Dependencies

C-02 (Pilot), C-05 (State), C-18 (Task Queue), System Bus.

Core Execution Code Snippet

```
// Logic for Code Reviewer AI (C-31)
async function executeCore_C_31(context, task) {
  const pilot = await Pilot.getChannel();
  const state = await StateCore.snapshot(task.id);

  // Core Logic: Static analysis → Sentiment check → Hint generation.
  const result = await Intelligence.process(core_definition, context);

  if (result.confidence > 0.95) {
    return await BuildEngine.commit(result.payload);
  } else {
    return await HumanLoop.trigger("VERIFY_OUTPUT", result);
  }
}
```

CORE MISSION

Legacy code modernization and clean-up.

INPUT SIGNAL	INTERNAL LOGIC FLOW	OUTPUT ARTIFACT
Context Vector + Intent	Pattern matching → Target mapping → Rewrite injection.	Validated Block

Dependencies

C-02 (Pilot), C-05 (State), C-18 (Task Queue), System Bus.

Core Execution Code Snippet

```
// Logic for Refactor AI (C-32)
async function executeCore_C_32(context, task) {
  const pilot = await Pilot.getChannel();
  const state = await StateCore.snapshot(task.id);

  // Core Logic: Pattern matching → Target mapping → Rewrite injection.
  const result = await Intelligence.process(core_definition, context);

  if (result.confidence > 0.95) {
    return await BuildEngine.commit(result.payload);
  } else {
    return await HumanLoop.trigger("VERIFY_OUTPUT", result);
  }
}
```

CORE MISSION

Decomposing monoliths into services.

INPUT SIGNAL	INTERNAL LOGIC FLOW	OUTPUT ARTIFACT
Context Vector + Intent	Dependency mapping → Domain isolation → Interface gen.	Validated Block

Dependencies

C-02 (Pilot), C-05 (State), C-18 (Task Queue), System Bus.

Core Execution Code Snippet

```
// Logic for Microservice Splitter (C-33)
async function executeCore_C_33(context, task) {
  const pilot = await Pilot.getChannel();
  const state = await StateCore.snapshot(task.id);

  // Core Logic: Dependency mapping → Domain isolation → Interface gen.
  const result = await Intelligence.process(core_definition, context);

  if (result.confidence > 0.95) {
    return await BuildEngine.commit(result.payload);
  } else {
    return await HumanLoop.trigger("VERIFY_OUTPUT", result);
  }
}
```


CORE MISSION

Smart caching layers (Redis/Browser).

INPUT SIGNAL	INTERNAL LOGIC FLOW	OUTPUT ARTIFACT
Context Vector + Intent	Invalidation logic → Hot-data detection → TTL manage.	Validated Block

Dependencies

C-02 (Pilot), C-05 (State), C-18 (Task Queue), System Bus.

Core Execution Code Snippet

```
// Logic for Cache Manager Core (C-34)
async function executeCore_C_34(context, task) {
  const pilot = await Pilot.getChannel();
  const state = await StateCore.snapshot(task.id);

  // Core Logic: Invalidation logic → Hot-data detection → TTL manage.
  const result = await Intelligence.process(core_definition, context);

  if (result.confidence > 0.95) {
    return await BuildEngine.commit(result.payload);
  } else {
    return await HumanLoop.trigger("VERIFY_OUTPUT", result);
  }
}
```

CORE MISSION

Tamper-proof system ledger.

INPUT SIGNAL	INTERNAL LOGIC FLOW	OUTPUT ARTIFACT
Context Vector + Intent	WORM storage → Chain hashing → Log rotation.	Validated Block

Dependencies

C-02 (Pilot), C-05 (State), C-18 (Task Queue), System Bus.

Core Execution Code Snippet

```
// Logic for Audit & Logging Core (C-35)
async function executeCore_C_35(context, task) {
  const pilot = await Pilot.getChannel();
  const state = await StateCore.snapshot(task.id);

  // Core Logic: WORM storage → Chain hashing → Log rotation.
  const result = await Intelligence.process(core_definition, context);

  if (result.confidence > 0.95) {
    return await BuildEngine.commit(result.payload);
  } else {
    return await HumanLoop.trigger("VERIFY_OUTPUT", result);
  }
}
```

CORE MISSION

Real-time CPU/GPU/Token telemetry.

INPUT SIGNAL	INTERNAL LOGIC FLOW	OUTPUT ARTIFACT
Context Vector + Intent	Polling → Alert thresholds → Scaling triggers.	Validated Block

Dependencies

C-02 (Pilot), C-05 (State), C-18 (Task Queue), System Bus.

Core Execution Code Snippet

```
// Logic for Resource Monitor Core (C-36)
async function executeCore_C_36(context, task) {
  const pilot = await Pilot.getChannel();
  const state = await StateCore.snapshot(task.id);

  // Core Logic: Polling → Alert thresholds → Scaling triggers.
  const result = await Intelligence.process(core_definition, context);

  if (result.confidence > 0.95) {
    return await BuildEngine.commit(result.payload);
  } else {
    return await HumanLoop.trigger("VERIFY_OUTPUT", result);
  }
}
```

CORE MISSION

Minimizing build spend via agent routing.

INPUT SIGNAL	INTERNAL LOGIC FLOW	OUTPUT ARTIFACT
Context Vector + Intent	Price prediction → Token optimization → Model switching.	Validated Block

Dependencies

C-02 (Pilot), C-05 (State), C-18 (Task Queue), System Bus.

Core Execution Code Snippet

```
// Logic for Cost Optimizer Core (C-37)
async function executeCore_C_37(context, task) {
  const pilot = await Pilot.getChannel();
  const state = await StateCore.snapshot(task.id);

  // Core Logic: Price prediction → Token optimization → Model switching.
  const result = await Intelligence.process(core_definition, context);

  if (result.confidence > 0.95) {
    return await BuildEngine.commit(result.payload);
  } else {
    return await HumanLoop.trigger("VERIFY_OUTPUT", result);
  }
}
```

CORE MISSION

Localized language support for apps.

INPUT SIGNAL	INTERNAL LOGIC FLOW	OUTPUT ARTIFACT
Context Vector + Intent	String extraction → Context translation → i18n injection.	Validated Block

Dependencies

C-02 (Pilot), C-05 (State), C-18 (Task Queue), System Bus.

Core Execution Code Snippet

```
// Logic for Translation AI (C-38)
async function executeCore_C_38(context, task) {
  const pilot = await Pilot.getChannel();
  const state = await StateCore.snapshot(task.id);

  // Core Logic: String extraction → Context translation → i18n injection.
  const result = await Intelligence.process(core_definition, context);

  if (result.confidence > 0.95) {
    return await BuildEngine.commit(result.payload);
  } else {
    return await HumanLoop.trigger("VERIFY_OUTPUT", result);
  }
}
```

CORE MISSION

Performance-based stress testing.

INPUT SIGNAL	INTERNAL LOGIC FLOW	OUTPUT ARTIFACT
Context Vector + Intent	Traffic simulation → Latency check → Bottleneck id.	Validated Block

Dependencies

C-02 (Pilot), C-05 (State), C-18 (Task Queue), System Bus.

Core Execution Code Snippet

```
// Logic for Benchmarking AI (C-39)
async function executeCore_C_39(context, task) {
  const pilot = await Pilot.getChannel();
  const state = await StateCore.snapshot(task.id);

  // Core Logic: Traffic simulation → Latency check → Bottleneck id.
  const result = await Intelligence.process(core_definition, context);

  if (result.confidence > 0.95) {
    return await BuildEngine.commit(result.payload);
  } else {
    return await HumanLoop.trigger("VERIFY_OUTPUT", result);
  }
}
```

CORE MISSION

Production uptime and status monitoring.

INPUT SIGNAL	INTERNAL LOGIC FLOW	OUTPUT ARTIFACT
Context Vector + Intent	Heartbeat polling → Endpoint ping → Incident logging.	Validated Block

Dependencies

C-02 (Pilot), C-05 (State), C-18 (Task Queue), System Bus.

Core Execution Code Snippet

```
// Logic for Health Check AI (C-40)
async function executeCore_C_40(context, task) {
  const pilot = await Pilot.getChannel();
  const state = await StateCore.snapshot(task.id);

  // Core Logic: Heartbeat polling → Endpoint ping → Incident logging.
  const result = await Intelligence.process(core_definition, context);

  if (result.confidence > 0.95) {
    return await BuildEngine.commit(result.payload);
  } else {
    return await HumanLoop.trigger("VERIFY_OUTPUT", result);
  }
}
```

CORE MISSION

Fine-tuning generated UX based on heatmaps.

INPUT SIGNAL	INTERNAL LOGIC FLOW	OUTPUT ARTIFACT
Context Vector + Intent	User track → Hotspot focus → UI adjustment.	Validated Block

Dependencies

C-02 (Pilot), C-05 (State), C-18 (Task Queue), System Bus.

Core Execution Code Snippet

```
// Logic for Refining Core (C-41)
async function executeCore_C_41(context, task) {
  const pilot = await Pilot.getChannel();
  const state = await StateCore.snapshot(task.id);

  // Core Logic: User track → Hotspot focus → UI adjustment.
  const result = await Intelligence.process(core_definition, context);

  if (result.confidence > 0.95) {
    return await BuildEngine.commit(result.payload);
  } else {
    return await HumanLoop.trigger("VERIFY_OUTPUT", result);
  }
}
```


CORE MISSION

Native vector storage for AI memory.

INPUT SIGNAL	INTERNAL LOGIC FLOW	OUTPUT ARTIFACT
Context Vector + Intent	Index management → Embedding update → Query optimize.	Validated Block

Dependencies

C-02 (Pilot), C-05 (State), C-18 (Task Queue), System Bus.

Core Execution Code Snippet

```
// Logic for Vector Database Engine (C-42)
async function executeCore_C_42(context, task) {
  const pilot = await Pilot.getChannel();
  const state = await StateCore.snapshot(task.id);

  // Core Logic: Index management → Embedding update → Query optimize.
  const result = await Intelligence.process(core_definition, context);

  if (result.confidence > 0.95) {
    return await BuildEngine.commit(result.payload);
  } else {
    return await HumanLoop.trigger("VERIFY_OUTPUT", result);
  }
}
```

CORE MISSION

OIDC/SAML/Social auth provider gen.

INPUT SIGNAL	INTERNAL LOGIC FLOW	OUTPUT ARTIFACT
Context Vector + Intent	Handshake logic → Token signing → Provider map.	Validated Block

Dependencies

C-02 (Pilot), C-05 (State), C-18 (Task Queue), System Bus.

Core Execution Code Snippet

```
// Logic for Authentication Logic Core (C-43)
async function executeCore_C_43(context, task) {
  const pilot = await Pilot.getChannel();
  const state = await StateCore.snapshot(task.id);

  // Core Logic: Handshake logic → Token signing → Provider map.
  const result = await Intelligence.process(core_definition, context);

  if (result.confidence > 0.95) {
    return await BuildEngine.commit(result.payload);
  } else {
    return await HumanLoop.trigger("VERIFY_OUTPUT", result);
  }
}
```

CORE MISSION

Image, video, and font processing.

INPUT SIGNAL	INTERNAL LOGIC FLOW	OUTPUT ARTIFACT
Context Vector + Intent	Optimization → CDN distribution → Lazy-load gen.	Validated Block

Dependencies

C-02 (Pilot), C-05 (State), C-18 (Task Queue), System Bus.

Core Execution Code Snippet

```
// Logic for Asset Pipeline (C-44)
async function executeCore_C_44(context, task) {
  const pilot = await Pilot.getChannel();
  const state = await StateCore.snapshot(task.id);

  // Core Logic: Optimization → CDN distribution → Lazy-load gen.
  const result = await Intelligence.process(core_definition, context);

  if (result.confidence > 0.95) {
    return await BuildEngine.commit(result.payload);
  } else {
    return await HumanLoop.trigger("VERIFY_OUTPUT", result);
  }
}
```

CORE MISSION

Real-time multi-human sync logic.

INPUT SIGNAL	INTERNAL LOGIC FLOW	OUTPUT ARTIFACT
Context Vector + Intent	Websocket management → cursor sync → intent merge.	Validated Block

Dependencies

C-02 (Pilot), C-05 (State), C-18 (Task Queue), System Bus.

Core Execution Code Snippet

```
// Logic for Collaboration Engine (C-45)
async function executeCore_C_45(context, task) {
  const pilot = await Pilot.getChannel();
  const state = await StateCore.snapshot(task.id);

  // Core Logic: Websocket management → cursor sync → intent merge.
  const result = await Intelligence.process(core_definition, context);

  if (result.confidence > 0.95) {
    return await BuildEngine.commit(result.payload);
  } else {
    return await HumanLoop.trigger("VERIFY_OUTPUT", result);
  }
}
```

CORE MISSION

Visualizing project progress and health.

INPUT SIGNAL	INTERNAL LOGIC FLOW	OUTPUT ARTIFACT
Context Vector + Intent	KPI calculation → Chart generation → Alert rollup.	Validated Block

Dependencies

C-02 (Pilot), C-05 (State), C-18 (Task Queue), System Bus.

Core Execution Code Snippet

```
// Logic for Project Dashboard AI (C-46)
async function executeCore_C_46(context, task) {
  const pilot = await Pilot.getChannel();
  const state = await StateCore.snapshot(task.id);

  // Core Logic: KPI calculation → Chart generation → Alert rollup.
  const result = await Intelligence.process(core_definition, context);

  if (result.confidence > 0.95) {
    return await BuildEngine.commit(result.payload);
  } else {
    return await HumanLoop.trigger("VERIFY_OUTPUT", result);
  }
}
```

CORE MISSION

In-app search logic and indexing.

INPUT SIGNAL	INTERNAL LOGIC FLOW	OUTPUT ARTIFACT
Context Vector + Intent	Index strategy → Crawler logic → Result ranking.	Validated Block

Dependencies

C-02 (Pilot), C-05 (State), C-18 (Task Queue), System Bus.

Core Execution Code Snippet

```
// Logic for Search Engine Core (C-47)
async function executeCore_C_47(context, task) {
  const pilot = await Pilot.getChannel();
  const state = await StateCore.snapshot(task.id);

  // Core Logic: Index strategy → Crawler logic → Result ranking.
  const result = await Intelligence.process(core_definition, context);

  if (result.confidence > 0.95) {
    return await BuildEngine.commit(result.payload);
  } else {
    return await HumanLoop.trigger("VERIFY_OUTPUT", result);
  }
}
```

CORE MISSION

Injecting user-feedback components.

INPUT SIGNAL	INTERNAL LOGIC FLOW	OUTPUT ARTIFACT
Context Vector + Intent	Modals → Surveys → Analytics hook injection.	Validated Block

Dependencies

C-02 (Pilot), C-05 (State), C-18 (Task Queue), System Bus.

Core Execution Code Snippet

```
// Logic for Feedback UI Core (C-48)
async function executeCore_C_48(context, task) {
  const pilot = await Pilot.getChannel();
  const state = await StateCore.snapshot(task.id);

  // Core Logic: Modals → Surveys → Analytics hook injection.
  const result = await Intelligence.process(core_definition, context);

  if (result.confidence > 0.95) {
    return await BuildEngine.commit(result.payload);
  } else {
    return await HumanLoop.trigger("VERIFY_OUTPUT", result);
  }
}
```

CORE MISSION

Version control for AI-build states.

INPUT SIGNAL	INTERNAL LOGIC FLOW	OUTPUT ARTIFACT
Context Vector + Intent	Diff storage → Revert logic → Branch management.	Validated Block

Dependencies

C-02 (Pilot), C-05 (State), C-18 (Task Queue), System Bus.

Core Execution Code Snippet

```
// Logic for Snapshot & Rollback Core (C-49)
async function executeCore_C_49(context, task) {
  const pilot = await Pilot.getChannel();
  const state = await StateCore.snapshot(task.id);

  // Core Logic: Diff storage → Revert logic → Branch management.
  const result = await Intelligence.process(core_definition, context);

  if (result.confidence > 0.95) {
    return await BuildEngine.commit(result.payload);
  } else {
    return await HumanLoop.trigger("VERIFY_OUTPUT", result);
  }
}
```


CORE MISSION

Production hardening and hand-off.

INPUT SIGNAL	INTERNAL LOGIC FLOW	OUTPUT ARTIFACT
Context Vector + Intent	Secret cleanup → Build optimization → Hand-off export.	Validated Block

Dependencies

C-02 (Pilot), C-05 (State), C-18 (Task Queue), System Bus.

Core Execution Code Snippet

```
// Logic for Finalization Engine (C-50)
async function executeCore_C_50(context, task) {
  const pilot = await Pilot.getChannel();
  const state = await StateCore.snapshot(task.id);

  // Core Logic: Secret cleanup → Build optimization → Hand-off export.
  const result = await Intelligence.process(core_definition, context);

  if (result.confidence > 0.95) {
    return await BuildEngine.commit(result.payload);
  } else {
    return await HumanLoop.trigger("VERIFY_OUTPUT", result);
  }
}
```

SECTION 4 — DATABASE & MEMORY PIPELINE

TRI-MODAL STORAGE ARCHITECTURE

ODEX operates on a state-synchronized tri-modal architecture to ensure infinite project horizontal growth while maintaining instant retrieval speeds.

1. RELATIONAL LEDGER (POSTGRESQL)

```
CREATE TABLE projects (  
  id UUID PRIMARY KEY,  
  owner_id UUID,  
  status build_status DEFAULT 'IDLE',  
  created_at TIMESTAMP,  
  config JSONB -- Stores the Pilot's strategic settings  
);  
  
CREATE TABLE core_logs (  
  core_id VARCHAR(10),  
  task_id UUID,  
  log_level VARCHAR(10),  
  payload JSONB,  
  trace_id UUID  
);
```

2. SEMANTIC MEMORY (VECTOR / RAG)

Uses **C-42 (Vector DB Engine)** to store every code snippet ever written for the user. When a new prompt comes in, the Pilot retrieves "Aesthetic Similarities" to ensure the new feature matches the style of old features.

SECTION 5 — THE ORCHESTRATOR & BUILD PIPELINE

TASK QUEUE & WORKER SYSTEM (C-18)

ODEX uses a priority-weighted, dependency-aware queue. If an "API" task depends on a "Database" task, the orchestrator holds the API worker in a **WAIT_STATE** until the DB heartbeat is confirmed.

```
// ORCHESTRATOR RETRY & DEPENDENCY LOGIC
class Orchestrator {
  async schedule(task) {
    const deps = await this.checkDependencies(task);
    if (deps.allResolved) {
      const worker = Swarm.getBrain(task.requiredCore);
      return worker.run(task).catch(e => this.retry(task, e));
    }
  }

  async retry(task, error) {
    task.retryCount++;
    if (task.retryCount < 3) {
      await wait(task.retryCount * 1000);
      return this.schedule(task);
    }
    return HumanLoop.alert("CRITICAL_TASK_FAILURE", task, error);
  }
}
```

SECTION 6 — LEARN & GO

BEHAVIOR CORE

This is the crowning feature of ODEX. The system is not static; it is a **Learning Entity**. Every human edit is treated as a "Fine-Tuning Signal."

THE FEEDBACK CYCLE

- **Capture:** Workspace monitors human edits in the Live IDE.
- **Analyze:** C-17 (Learn & Go) compares AI output vs. Human correction.
- **Distill:** System abstracts the preference (e.g., "User prefers 8px border-radius over 4px").
- **Apply:** The preference is stored in the User Style Vector, affecting all future C-11 (UI) and C-07 (Code) generations.

ODEX SYSTEM // ALPHA-01

BLUEPRINT ARCHIVE COMPLETE. END OF SPECIFICATION.